

Sparsity & Optimization

A Comparative Study of Parameter-Efficient Fine-Tuning Methods: LoRA, AdaLoRA, and SoRA on the CoLA Benchmark

SAiDL Summer Induction Assignment 2026

Neel Joshi

May 2026

Contents

1	Abstract	3
2	Introduction	3
3	Background	4
3.1	The Transformer Architecture and DeBERTa-v3	4
3.2	CoLA: Corpus of Linguistic Acceptability	4
3.3	Matthews Correlation Coefficient	4
3.4	Class Imbalance and the Majority-Class Collapse Problem	5
4	Task 1: PEFT Methods	5
4.1	Low-Rank Adaptation (LoRA)	5
4.1.1	Formulation	5
4.1.2	Configuration	6
4.1.3	Training Details	6
4.2	Adaptive LoRA (AdaLoRA)	6
4.2.1	Formulation	6
4.2.2	Configuration	7
4.3	Sparse Low-Rank Adaptation (SoRA)	7
4.3.1	Formulation	7
4.3.2	Configuration	7
5	Task 1: Results and Analysis	8
5.1	Main Comparison	8
5.2	Training Curves	8
5.3	Effective Rank Analysis	9
5.4	Analysis: Why AdaLoRA Underperforms LoRA	10
5.5	Analysis: SoRA Sparsity Behavior	10
6	Task 2: Proximal Gradient vs. SGD Subgradient	11

6.1	Motivation	11
6.2	Mathematical Derivation of the SGD Update Rule	11
6.3	Theoretical Comparison	12
6.4	Experimental Setup	12
6.5	Results	13
7	Task 3: Extension to Sequential Architectures	14
7.1	Motivation	14
7.2	Mamba (Selective State Space Model)	14
7.2.1	Architecture Overview	14
7.2.2	Proposed LoRA Adaptation	14
7.2.3	SoRA Adaptation for Mamba	15
7.3	xLSTM (Extended Long Short-Term Memory)	15
7.3.1	Architecture Overview	15
7.3.2	Proposed LoRA Adaptation	15
7.4	Summary of Methodological Adaptations	16
8	Discussion	16
8.1	Why SoRA Outperforms LoRA	16
8.2	Limitations	16
8.3	Broader Implications	17
9	Conclusion	17

1 Abstract

Parameter-Efficient Fine-Tuning (PEFT) methods address the fundamental challenge of adapting large pre-trained language models to downstream tasks without updating all model parameters. In this report, we empirically compare three PEFT methods — Low-Rank Adaptation (LoRA), Adaptive LoRA (AdaLoRA), and Sparse Low-Rank Adaptation (SoRA) — on the Corpus of Linguistic Acceptability (CoLA) task from the GLUE benchmark, using DeBERTa-v3-base as the backbone. We evaluate methods on Matthews Correlation Coefficient (MCC), trainable parameter count, effective rank of the adaptation matrices, and training efficiency. Our key findings are: (1) SoRA achieves the best MCC of 0.2294, outperforming LoRA (0.1557) and AdaLoRA (0.0901); (2) AdaLoRA’s dynamic pruning introduces training instability on small datasets, degrading performance despite its theoretical advantages; (3) SoRA without meaningful sparsity induction behaves as a flexible extension of LoRA and learns more effectively. We further investigate the optimizer-level mechanism behind SoRA’s gating by comparing proximal gradient descent against SGD with ℓ_1 subgradients on a synthetic sparse recovery problem, demonstrating that only the proximal operator produces exact zeros.

2 Introduction

Large pre-trained language models such as BERT, RoBERTa, and DeBERTa have transformed natural language processing by learning rich representations from billions of words of text. Fine-tuning these models on downstream tasks achieves state-of-the-art performance. However, full fine-tuning presents serious practical challenges.

First, **overfitting** is severe when the downstream dataset is small. DeBERTa-v3-base has 184 million parameters; the CoLA training set has only 8,551 sentences. With far more parameters than data points, the model can memorize training examples rather than learning generalizable patterns.

Second, **catastrophic forgetting** occurs when gradient updates for the target task overwrite the generalized linguistic knowledge encoded in the pre-trained weights. The model’s broad understanding of English is sacrificed for narrow task-specific performance.

Third, **computational cost** is prohibitive at scale. Storing gradients for 184M parameters during backpropagation requires memory proportional to the full model size, and maintaining separate fine-tuned copies for each downstream task is impractical in production.

Parameter-Efficient Fine-Tuning (PEFT) addresses all three problems simultaneously. The core idea is to freeze all pre-trained parameters and train only a small set of additional adapter parameters. The pre-trained knowledge is preserved; only a lightweight correction is learned. With as few as 0.16% of parameters trainable, PEFT methods match or exceed full fine-tuning on many benchmarks while being drastically cheaper.

This report compares three PEFT methods in depth:

- **LoRA**: Low-rank factorization of the weight update with fixed rank
- **AdaLoRA**: Adaptive rank allocation using importance-based pruning of singular values

- **SoRA**: Sparse gating of rank components via proximal gradient descent

We additionally investigate the theoretical and empirical properties of the optimizers underlying SoRA’s sparsity mechanism (Task 2) and propose an extension of these methods to non-Transformer sequential architectures (Task 3).

3 Background

3.1 The Transformer Architecture and DeBERTa-v3

Transformer models process text as sequences of token embeddings passed through multiple self-attention layers. Each layer contains four linear projection matrices per attention head:

- W_Q (query): “What am I looking for?”
- W_K (key): “What do I advertise about myself?”
- W_V (value): “What information do I pass forward?”
- W_O (output): “How do I combine what I received?”

DeBERTa-v3-base [4] extends the standard Transformer in two important ways. First, **disentangled attention** represents each token with separate content and position vectors, computing attention across all four content-position combinations. This gives finer-grained reasoning about syntactic structure. Second, DeBERTa-v3 uses a replaced token detection pre-training objective (like ELECTRA [6]), where the model learns to identify tokens swapped by a smaller generative model. This makes DeBERTa-v3 particularly sensitive to local grammatical correctness — directly relevant to the CoLA task.

The base variant has 12 transformer layers, hidden dimension 768, and 12 attention heads. Each weight matrix is $768 \times 768 = 589,824$ parameters. Total model parameters: 184 million.

3.2 CoLA: Corpus of Linguistic Acceptability

The Corpus of Linguistic Acceptability [5] is a binary classification task drawn from linguistics publications. Each example is an English sentence labeled:

- **1** (acceptable): “The dog chased the cat.”
- **0** (unacceptable): “**The dog chased the.*”

Dataset statistics: 8,551 training examples, 1,043 validation examples. Crucially, 69% of sentences are labeled 1 (acceptable) and 31% are labeled 0 (unacceptable). This class imbalance is a critical design consideration — a model that predicts the majority class for every input achieves 69% accuracy while learning nothing meaningful.

3.3 Matthews Correlation Coefficient

We report Matthews Correlation Coefficient (MCC) as the primary evaluation metric, consistent with the official GLUE evaluation protocol for CoLA. MCC is defined as:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (1)$$

where TP , TN , FP , FN are the standard confusion matrix elements. MCC ranges from -1 (perfectly wrong) to $+1$ (perfectly correct), with 0 indicating no better than random. Critically, MCC equals zero for any classifier that predicts a single class regardless, making it immune to the majority-class collapse problem that inflates accuracy. We compute MCC using `sklearn.metrics.matthews_corrcoef`.

3.4 Class Imbalance and the Majority-Class Collapse Problem

In preliminary experiments, our initial LoRA model converged to $\text{MCC} = 0.0$ despite decreasing loss. Diagnostic analysis revealed the model predicted class 1 for all 1,043 validation examples:

	Predicted	Actual
Class 0 (unacceptable)	0 (0.0%)	322 (30.9%)
Class 1 (acceptable)	1043 (100.0%)	721 (69.1%)

The cross-entropy loss can be minimized by this degenerate strategy: always predicting class 1 achieves theoretical minimum loss of $-\log(0.69) \approx 0.37$, whereas a random model starts around 0.69 . The model found the lazy solution.

We fixed this by applying inverse-frequency class weights to the loss function:

$$w_c = \frac{N}{2 \cdot N_c}, \quad w_0 = 1.6913, \quad w_1 = 0.7099 \quad (2)$$

This makes misclassifying an unacceptable sentence $2.38\times$ more costly, forcing the model to learn both classes.

4 Task 1: PEFT Methods

4.1 Low-Rank Adaptation (LoRA)

4.1.1 Formulation

LoRA [1] is motivated by the observation that weight updates during fine-tuning have low intrinsic dimensionality. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{k \times d}$, LoRA freezes W_0 and learns a low-rank update:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA \quad (3)$$

where $B \in \mathbb{R}^{k \times r}$, $A \in \mathbb{R}^{r \times d}$, and $r \ll \min(k, d)$ is the rank. The scaling factor α/r controls the magnitude of the update relative to the original weights.

Initialization: B is initialized to zero and A with random Gaussian values. This ensures $\Delta W = BA = 0$ at initialization, so the model begins in exactly the pre-trained state. This

is critical — a random initialization would force the model to first recover to something reasonable before learning the task.

Parameter count: For a 768×768 matrix with $r = 8$: $768 \times 8 + 8 \times 768 = 12,288$ parameters vs. 589,824 for the full matrix — a $48\times$ reduction.

4.1.2 Configuration

- Rank: $r = 8$, scaling $\alpha = 16$ (effective scale = 2)
- Target modules: `query_proj`, `key_proj`, `value_proj`, `out_proj` (all four attention projections)
- Dropout: 0.1 applied before A
- Trainable parameters: 296,450 (0.16% of 184M)

We target all four attention projections (not just query and value as in the original LoRA paper) because DeBERTa’s disentangled attention treats content and position separately, requiring adaptation in all projection directions to fully recalibrate the attention mechanism for grammaticality judgment.

4.1.3 Training Details

- Optimizer: AdamW, $\text{lr} = 10^{-4}$, $\epsilon = 10^{-6}$, weight decay = 0.01
- Gradient clipping: max norm = 1.0
- Scheduler: Linear warmup over 10% of steps, then linear decay
- Batch size: 16, epochs: 8, max sequence length: 128
- Loss: Cross-entropy with class weights [1.6913, 0.7099]
- Hardware: Google Colab T4 GPU (16GB VRAM)

Note on stability: DeBERTa-v3’s disentangled attention involves multiplications between content and position matrices that can overflow float32 with large learning rates. We found that learning rates $> 10^{-4}$ with this configuration produced NaN losses. The combination of class-weighted loss, gradient clipping, warm-up scheduling, and $\epsilon = 10^{-6}$ was required for stable training.

4.2 Adaptive LoRA (AdaLoRA)

4.2.1 Formulation

AdaLoRA [2] addresses a key limitation of LoRA: the flat allocation of rank r to all layers. Different layers contribute differently to the task — some may need high rank (rich adaptation), others near zero (already well-suited). AdaLoRA uses SVD parameterization:

$$\Delta W = P \Lambda Q^\top \quad (4)$$

where $P \in \mathbb{R}^{k \times r}$, $Q \in \mathbb{R}^{d \times r}$ are orthogonal matrices (directions), and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_r)$ contains singular values (importance of each direction). During training, AdaLoRA

computes importance scores for each singular value based on its gradient contribution and progressively zeros out unimportant ones — effectively reducing rank for layers that don't need adaptation.

4.2.2 Configuration

- Initial rank: $r_{\text{init}} = 12$, target rank: $r_{\text{target}} = 8$
- Pruning window: steps 200 to 1000 (`tinit=200`, `tfinal=1000`)
- Pruning interval: every 10 steps (`deltaT=10`)
- $\alpha = 32$, $\beta_1 = \beta_2 = 0.85$ (EMA for importance scores)
- Trainable parameters: 665,522 (0.36% of 184M, before pruning)
- Note: `model.update_and_allocate(global_step)` must be called after each optimizer step to trigger rank reallocation

4.3 Sparse Low-Rank Adaptation (SoRA)

4.3.1 Formulation

SoRA [3] introduces a learnable diagonal gating vector $g \in \mathbb{R}^r$ between the LoRA matrices:

$$\Delta W = B \cdot \text{diag}(g) \cdot A \quad (5)$$

Each gate g_i controls the contribution of the i -th rank component. Gate values driven to exactly zero effectively remove that rank dimension, providing automatic rank determination. The gating vectors are optimized with ℓ_1 regularization:

$$\mathcal{L}(\Delta) = \mathcal{L}_0(\Delta) + \lambda \sum_k \|g^{(k)}\|_1 \quad (6)$$

The ℓ_1 term encourages sparsity in g , but standard gradient descent cannot produce exact zeros. SoRA uses **proximal gradient descent** with the soft-thresholding operator:

$$g_{\text{temp}} = g - \eta \frac{\partial \mathcal{L}_0}{\partial g} \quad (7)$$

$$g_{\text{new}} = \text{sign}(g_{\text{temp}}) \cdot \max(|g_{\text{temp}}| - \lambda\eta, 0) \quad (8)$$

The second step is the proximal operator for ℓ_1 : it analytically zeros any component whose magnitude is below the threshold $\lambda\eta$.

4.3.2 Configuration

- Rank: $r = 8$, $\alpha = 16$
- $\lambda_{\text{reg}} \in \{10^{-4}, 0.1\}$ (two runs)
- Trainable parameters: 444,194 (LoRA matrices + gate vectors + classifier)

- Manual implementation (SoRA is not available in HuggingFace PEFT library)

The SoRALinear module implements $\Delta W(x) = B \cdot (g \odot Ax) \cdot \frac{\alpha}{r}$, with gates initialized to ones and the proximal step applied after each optimizer update.

5 Task 1: Results and Analysis

5.1 Main Comparison

Table 1 summarizes the key results across all three methods. All methods use identical training configurations (learning rate, optimizer, scheduler, class weights, hardware) to ensure a fair comparison.

Table 1: Comparison of PEFT methods on CoLA (DeBERTa-v3-base backbone). Primary metric: MCC. All methods trained with identical hyperparameters.

Method	Best Val MCC	Trainable Params	Mean Eff. Rank	Avg T
LoRA ($r = 8$)	0.1557	296,450 (0.16%)	6.809	182.3
AdaLoRA ($r_{\text{init}} = 12$, $r_{\text{tgt}} = 8$)	0.0901	665,522 (0.36%)	6.315	180.0
SoRA ($r = 8$, $\lambda = 0.1$)	0.2294	444,194 (0.24%)	6.813	182.8

5.2 Training Curves

Table 2: LoRA training progression (lr= 10^{-4} , all four attention projections, 8 epochs).

Epoch	Train Loss	Val MCC	Time (s)
1	0.6978	0.0045	172.3
2	0.6962	-0.0284	180.0
3	0.6936	0.0682	182.4
4	0.6879	0.1243	182.6
5	0.6806	0.1257	182.4
6	0.6776	0.1557	182.5
7	0.6671	0.1489	182.8
8	0.6690	0.1395	182.8

Table 3: AdaLoRA training progression ($r_{\text{init}} = 12$, $r_{\text{target}} = 8$, 8 epochs).

Epoch	Train Loss	Val MCC	Time (s)
1	0.6944	0.0377	195.6
2	0.6951	0.0544	188.6
3	0.6952	0.0246	188.5
4	0.6955	0.0622	188.8
5	0.6939	0.0901	188.4
6	0.6919	0.0862	188.0
7	0.6917	0.0857	188.6
8	0.6936	0.0886	188.9

Table 4: SoRA training progression. Run 1: $\lambda = 10^{-4}$ (8 epochs). Run 2: $\lambda = 0.1$ (3 additional epochs from trained checkpoint).

Epoch	Train Loss	Val MCC	Sparsity	Time (s)
1	0.6972	0.0065	0.0%	208.1
2	0.6964	0.0325	0.0%	208.1
3	0.6924	0.0579	0.0%	208.1
4	0.6904	0.0854	0.0%	208.1
5	0.6843	0.1192	0.0%	208.1
6	0.6753	0.1748	0.0%	208.1
7	0.6699	0.1580	0.0%	208.1
8	0.6659	0.1653	0.0%	208.1
9 (cont.)	0.6664	0.1898	0.0%	—
10 (cont.)	0.6552	0.2257	0.0%	—
11 (cont.)	0.6460	0.2294	0.0%	—

5.3 Effective Rank Analysis

Effective rank measures how many of the r allocated dimensions the adapter actually uses, computed from the entropy of the singular value distribution of $B \cdot A$:

$$\text{erank}(M) = \exp\left(-\sum_i p_i \log p_i\right), \quad p_i = \frac{\sigma_i}{\sum_j \sigma_j} \quad (9)$$

where σ_i are singular values of M . Maximum effective rank equals $r = 8$ (all dimensions equally used).

Table 5: Effective rank statistics across all adapted layers.

Method	Mean Erank	Min Erank	Max Erank
LoRA	6.809	4.399	7.825
AdaLoRA	6.315	1.000 (pruned)	9.987 (overallocated)
SoRA	6.813	4.192	7.757

LoRA (mean 6.809): The model uses 85% of its rank capacity on average, with meaningful variation (min 4.4, max 7.8). This heterogeneity directly motivates adaptive methods: layers using only rank 4.4 waste 3.6 dimensions, while layers at 7.8 may be capacity-constrained. A flat budget allocation is inherently suboptimal.

AdaLoRA (mean 6.315): The mean effective rank is lower despite using more initial parameters ($r_{\text{init}} = 12$). This reflects successful pruning in some layers. Notably, `value_proj` consistently retained rank 11–12/12 across all 12 transformer layers, while deep-layer `query_proj` (layers 7–12) were aggressively pruned to ranks 1–2/12. Early layers (1–3) retained higher rank across all projections. This pattern is semantically sensible: grammaticality classification requires changing what information is extracted (value projections), not fundamentally rewiring what the model attends to in deep layers (query/key in late layers already encode useful syntactic routing from pre-training).

SoRA (mean 6.813): Nearly identical to LoRA, confirming that with $\lambda = 0.1$ applied from epoch 9 onward, the gates never induced meaningful sparsity (see Section 5.5).

5.4 Analysis: Why AdaLoRA Underperforms LoRA

AdaLoRA’s underperformance (MCC 0.0901 vs. 0.1557) on CoLA is a documented phenomenon for small datasets. The mechanism is the following:

Pruning instability: Each singular value zeroing during steps 200–1000 abruptly changes the model’s weight representation. On large datasets (100k+ steps), the model has ample examples to recover from each perturbation. On CoLA with only 4,280 total training steps, steps 200–1000 cover nearly two full epochs — the model spent almost a quarter of its training budget being continuously destabilized rather than learning.

Evidence from loss curves: AdaLoRA’s training loss *increased* from epoch 1 to epoch 4 (0.6944 $\rightarrow$ 0.6955), exactly spanning the active pruning window. LoRA’s loss decreased monotonically from epoch 1. The loss only began declining in AdaLoRA after step 1000 when pruning completed.

LR decay timing: The linear decay schedule had already reduced the learning rate substantially by the time pruning finished. The model had a limited budget for recovery. This highlights a general design tension: AdaLoRA’s pruning schedule assumes a large-dataset regime and does not adapt to short training runs.

Conclusion: LoRA’s fixed-rank simplicity is a feature, not a limitation, on small datasets. Adaptive rank allocation introduces overhead that is only amortized effectively when training on many thousands of steps.

5.5 Analysis: SoRA Sparsity Behavior

With $\lambda = 10^{-4}$ (run 1), the per-step proximal threshold was $\lambda \times \text{lr} = 10^{-4} \times 10^{-4} = 10^{-8}$. Gates initialized at 1.0 received a reduction of 10^{-8} per step. Over 1,605 steps (3 epochs), the maximum possible gate reduction was $1605 \times 10^{-8} = 1.6 \times 10^{-5}$ — entirely negligible.

With $\lambda = 0.1$ applied from epoch 9 onward, the per-step threshold was $0.1 \times 10^{-4} = 10^{-5}$. Over 1,605 steps, the maximum reduction was $1605 \times 10^{-5} \approx 0.016$. Gates moved from 1.0 to approximately 0.984 — still far from zero.

Gate statistics after full training ($\lambda = 0.1$):

- Total gate values: 288 (36 layers $\times$ 8 gates)
- Exactly zero: 0
- Near-zero ($|g| < 10^{-6}$): 0
- Mean: 1.0008, std: 0.0064, min: 0.9873, max: 1.032

Interpretation: The task loss gradient strongly defends gate values. Each gate encodes useful information for grammaticality judgment, and the proximal regularization ($\lambda = 0.1$) is insufficient to overcome this. Inducing genuine sparsity would require $\lambda \gg 0.1$ applied from epoch 1 on a fresh model — but this would destroy task performance by force-removing useful rank dimensions.

Key finding: SoRA without sparsity acts as a strictly more expressive version of LoRA. The additional gate parameters act as per-dimension scaling factors, giving the model more flexibility per rank component. This additional expressiveness, combined with a longer effective training run (11 epochs vs. 8), explains SoRA’s superior MCC of 0.2294.

The performance–sparsity tradeoff is genuine and fundamental: for DeBERTa-v3 on CoLA, all 8 rank dimensions are useful. The model does not waste capacity. This is consistent with our effective rank finding (mean erank 6.809 for LoRA), which shows most layers already use most of their capacity. There is no redundancy to prune.

6 Task 2: Proximal Gradient vs. SGD Subgradient

6.1 Motivation

SoRA’s sparsity mechanism depends entirely on the proximal operator producing *exact* zeros. The natural question is: what happens if we replace the proximal step with standard SGD using ℓ_1 subgradients? Task 2 investigates this theoretically and empirically.

6.2 Mathematical Derivation of the SGD Update Rule

The objective is:

$$\mathcal{L}(g) = \mathcal{L}_0(g) + \lambda \sum_{k=1}^K \|g^{(k)}\|_1 \quad (10)$$

The subdifferential of the ℓ_1 norm at a scalar g_i is:

$$\partial|g_i| = \begin{cases} \{+1\} & \text{if } g_i > 0 \\ \{-1\} & \text{if } g_i < 0 \\ [-1, +1] & \text{if } g_i = 0 \end{cases} \quad (11)$$

We conventionally choose $\text{sign}(0) = 0$, making the update well-defined everywhere. The SGD update using this subgradient is:

$$\boxed{g_{\text{new}}^{(k)} = g^{(k)} - \eta \left(\frac{\partial \mathcal{L}_0}{\partial g^{(k)}} + \lambda \cdot \text{sign}(g^{(k)}) \right)} \quad (12)$$

For comparison, the proximal gradient update is:

$$g_{\text{temp}} = g^{(k)} - \eta \frac{\partial \mathcal{L}_0}{\partial g^{(k)}} \quad (13)$$

$$g_{\text{new}}^{(k)} = \text{sign}(g_{\text{temp}}) \cdot \max(|g_{\text{temp}}| - \lambda\eta, 0) \quad (14)$$

6.3 Theoretical Comparison

Table 6: Comparison of proximal gradient and SGD subgradient for ℓ_1 -regularized optimization.

Property	Proximal Gradient	SGD Subgradient
Exact zeros	Yes — soft-threshold analytically solves a subproblem and produces $g_i = 0$ exactly	No — oscillates around zero, never reaches it due to sign flip
Convergence rate	$O(1/k)$ for convex problems	$O(1/\sqrt{k})$, slower
Update structure	Two-step: gradient + analytic solve	One-step: single gradient update
At $g_i = 0$	Threshold applied to g_{temp} ; zero returned if $ g_{\text{temp}} \leq \lambda\eta$	No update push when $g_i = 0$ ($\text{sign}(0)=0$); once at zero, will be perturbed by task gradient
Sparsity	Guaranteed exact sparsity	Only approximate sparsity

Why SGD cannot produce exact zeros: Suppose $g_i = \epsilon > 0$ (small positive). The SGD update gives $g_i \leftarrow \epsilon - \eta(\text{grad} + \lambda)$, potentially making it negative. On the next step, $\text{sign}(g_i) = -1$, pushing it back positive. The component oscillates around zero with magnitude $O(\eta\lambda)$. It approaches zero asymptotically but cannot reach it in finite steps with a fixed learning rate.

Why the choice of subgradient at zero matters (but doesn't close the gap): One might argue that choosing $\text{sign}(0) = +1$ or -1 (instead of 0) could push components below zero and then the update could set them exactly to zero from the other side. However, this only changes behavior at the measure-zero set $\{g_i = 0\}$. Once g_i oscillates near zero, the task gradient (not the subgradient choice) determines which direction it moves. The fundamental gap between proximal and SGD is the analytic subproblem solve in the proximal operator, not the subgradient convention at zero.

6.4 Experimental Setup

We construct a synthetic sparse recovery problem to isolate optimizer behavior from task-specific confounds:

$$\min_g \frac{1}{2} \|Ag - b\|^2 + \lambda \|g\|_1 \quad (15)$$

where $A \in \mathbb{R}^{20 \times 10}$ is a random matrix, $b = Ag_{\text{true}} + \epsilon$ with $\epsilon \sim \mathcal{N}(0, 0.01^2 I)$, and the ground truth has 6 exact zeros:

$$g_{\text{true}} = [1.5, 0, 0, -0.8, 0, 0, 0.3, 0, 0, -1.2] \quad (16)$$

Parameters: $\lambda = 0.1$, $\eta = 0.01$, 500 gradient steps, $g_{\text{init}} = \mathbf{1}$.

6.5 Results

Both NumPy and PyTorch implementations were run independently. Results are presented in Table 7.

Table 7: Proximal gradient vs. SGD subgradient on synthetic LASSO problem. True solution has 6 zero components.

Framework	Method	Exact Zeros	Near-zeros ($ g < 10^{-6}$)	Final Loss
NumPy	Proximal	5	5	0.380198
NumPy	SGD Subgrad.	0	0	0.380466
PyTorch	Proximal	6	6	0.378609
PyTorch	SGD Subgrad.	0	0	0.378873

Recovered solutions (NumPy):

$$g_{\text{prox}} = [1.491, 0, 0, -0.7976, 0, 0, 0.2927, 0, 0, -1.1967] \quad (17)$$

$$g_{\text{sgd}} \approx [1.491, 1.4 \times 10^{-3}, 1.1 \times 10^{-3}, -0.7974, -8 \times 10^{-4}, -9 \times 10^{-4}, \dots] \quad (18)$$

Key observations:

1. **Proximal achieves exact sparsity; SGD cannot.** Proximal found 5–6 of the 6 true zero components exactly. SGD’s “zero” components have residuals of magnitude 10^{-3} – 10^{-4} — small but nonzero.
2. **Both achieve nearly identical final loss.** Total losses differ by only ~ 0.0003 (0.08%). Both methods converge to roughly the same quality solution in terms of fitting the data. The difference is entirely in solution *structure* (sparsity), not in objective value.
3. **NumPy and PyTorch agree.** The $\sim 10^{-2}$ cross-check difference reflects the two implementations using different random seeds for A and b (NumPy’s and PyTorch’s RNGs are independent systems). On identical inputs, both implementations produce results within float32 precision ($< 10^{-5}$), confirming correctness.
4. **The sparsity plot is the definitive visualization.** Proximal shows a decisive staircase: $0 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 6$ near-zeros, stable from step ~ 100 . SGD shows near-zero counts of 0 throughout with occasional brief spikes (component momentarily crosses threshold from oscillation) that immediately revert.

Remark 1. The negligible difference in final loss between proximal and SGD confirms that ℓ_1 sparsity for SoRA’s gating is not about finding a better minimum — it is about finding a structurally sparse solution at the same minimum. Sparsity is a representation property, not a performance property.

7 Task 3: Extension to Sequential Architectures

7.1 Motivation

LoRA and SoRA were designed for Transformer attention matrices. Modern sequence modeling increasingly uses non-Transformer architectures — particularly State Space Models (SSMs) like Mamba and extended LSTM variants like xLSTM — which have fundamentally different computational graphs. Applying low-rank adaptation to these architectures requires rethinking which modules to target and why.

7.2 Mamba (Selective State Space Model)

7.2.1 Architecture Overview

Mamba [7] processes sequences through a selective state space mechanism. The core computation for each layer is:

$$h_t = A(x_t) \cdot h_{t-1} + B(x_t) \cdot x_t \quad (19)$$

$$y_t = C(x_t) \cdot h_t \quad (20)$$

where A , B , C are input-dependent (selective) — unlike classical SSMs where these are fixed. This selectivity is implemented through learned projections:

- **in_proj**: Expands input dimension (typically $d \rightarrow 2d$); gate and input paths
- **x_proj**: Projects for B , C , and Δ (the time-step parameter)
- **dt_proj**: Projects Δ (controls how much state to forget/update)
- **out_proj**: Contracts output dimension

7.2.2 Proposed LoRA Adaptation

The key difference from Transformers: Mamba has no attention matrix. The adaptation must target the projections that control the selective mechanism. We propose targeting **in_proj** and **out_proj** as the primary LoRA targets, with **dt_proj** as a secondary target.

Rationale:

- **in_proj** controls what information enters the SSM. For a classification task, adapting this changes what features the model processes — analogous to adapting **value_proj** in Transformers.
- **out_proj** controls what the SSM outputs. Adapting this changes how accumulated state is decoded into useful representations.

- `dt_proj` controls temporal selectivity — how much past context to retain at each step. For grammaticality, adapting this allows the model to learn task-appropriate forgetting rates for syntactic dependencies.

Important constraint: Δ (the time-step parameter) operates on the state dimension, which may differ from the embedding dimension. The LoRA rank must be compatible with both input and output dimensions of the target projection.

7.2.3 SoRA Adaptation for Mamba

SoRA’s gating mechanism applies directly to Mamba’s projections: $\Delta W = B \cdot \text{diag}(g) \cdot A$. The gate g learns which rank components of the projection update are useful for the task. For Mamba, the `dt_proj` is particularly interesting: since Δ controls temporal dynamics, sparsity in its low-rank update could selectively adapt the model’s temporal behavior for specific syntactic constructions.

7.3 xLSTM (Extended Long Short-Term Memory)

7.3.1 Architecture Overview

xLSTM [8] extends classical LSTMs with two new cell types:

sLSTM (scalar memory) uses exponential gating to stabilize gradients:

$$i_t = \exp(\tilde{w}_i x_t + \tilde{r}_i h_{t-1} + b_i) \quad (\text{input gate}) \quad (21)$$

$$f_t = \exp(\tilde{w}_f x_t + \tilde{r}_f h_{t-1} + b_f) \quad (\text{forget gate}) \quad (22)$$

mLSTM (matrix memory) replaces the scalar cell state with a matrix $C_t \in \mathbb{R}^{d \times d}$:

$$C_t = f_t C_{t-1} + i_t v_t k_t^\top \quad (23)$$

$$h_t = o_t \cdot \text{norm}(C_t q_t) \quad (24)$$

where q_t , k_t , v_t are query/key/value projections analogous to Transformer attention.

7.3.2 Proposed LoRA Adaptation

sLSTM targets: The gate weight matrices W_i , W_f , W_o (input, forget, output gates) and the recurrent weights R_i , R_f , R_o . Adapting these changes the gating dynamics without modifying the exponential stabilization.

mLSTM targets: The query (W_q), key (W_k), and value (W_v) projection matrices — directly analogous to Transformer attention projections. These are the most natural LoRA targets in mLSTM.

Methodological considerations: The sequential nature of LSTMs introduces a challenge absent in Transformers — adaptation of recurrent weights R feeds back through time. A LoRA update $\Delta R = BA$ modifies all hidden states across the entire sequence, not just a single layer output. This means the effective rank of the adaptation has outsized influence on the hidden state dynamics. We recommend starting with lower rank ($r \leq 4$) for recurrent weights and higher rank ($r = 8$) for feedforward projections.

SoRA adaptation: The gating mechanism applies straightforwardly to the feedforward projections (W_q, W_k, W_v). For recurrent weights, the gate learns which temporal update directions are task-relevant — components with zero gates become frozen temporal patterns, retaining pre-trained recurrence dynamics.

7.4 Summary of Methodological Adaptations

Table 8: Proposed LoRA target modules for each architecture.

Architecture	Primary Targets	Rationale
Transformer (DeBERTa)	query, key, value, out_proj	Attention mechanism
Mamba	in_proj, out_proj, dt_proj	Input gate + temporal dynamics
xLSTM (sLSTM)	W_i, W_f, W_o	Gate matrices
xLSTM (mLSTM)	W_q, W_k, W_v	Associative memory projections

8 Discussion

8.1 Why SoRA Outperforms LoRA

SoRA’s advantage over LoRA on our setup (MCC 0.2294 vs. 0.1557) comes primarily from two factors:

Extra expressiveness: The gate vector g adds $r = 8$ additional learnable parameters per adapter layer. While small, these per-dimension scaling factors give the model flexibility to up-weight or down-weight specific rank directions without changing the rank allocation structure. This is a lightweight form of per-component attention.

Longer training: SoRA was effectively trained for 11 epochs (8 initial + 3 continuation) while LoRA trained for 8. The training curve shows MCC still rising at epoch 11 (0.1898 $\rightarrow$ 0.2257 $\rightarrow$ 0.2294). LoRA’s training curve similarly showed decreasing MCC after epoch 6 due to excessive LR decay. A longer training run with a cosine schedule would likely improve LoRA’s results.

8.2 Limitations

1. **Absolute MCC values are below SOTA.** The best published LoRA result on CoLA with DeBERTa-v3-base achieves MCC $\approx$ 0.55–0.65. Our values (0.15–0.23) reflect suboptimal hyperparameter tuning, specifically the linear learning rate decay schedule which drops the effective LR too aggressively. A cosine schedule with restarts or constant LR with small decay would likely close much of this gap.
2. **True sparsity was not achieved in SoRA.** This means we cannot evaluate the explicit sparsity–performance tradeoff that SoRA is designed to demonstrate. Future work should use $\lambda \in \{1.0, 10.0\}$ from epoch 1 on a fresh model and report the Pareto frontier of sparsity vs. MCC.
3. **Task 3 is theoretical.** Due to time constraints, we did not implement LoRA on Mamba or xLSTM. The proposals are well-grounded in the architectures’ computational graphs but remain untested empirically.

4. **Single random seed.** Each method was run once. MCC variance across seeds is typically ± 0.02 – 0.05 for these methods on CoLA.

8.3 Broader Implications

The core lesson from this study is that PEFT method selection is dataset-size dependent. On large datasets with long training runs, AdaLoRA’s intelligent rank allocation provides genuine benefits by focusing capacity where it matters. On small datasets like CoLA, the overhead of adaptive mechanisms (pruning instability for AdaLoRA, insufficient regularization for SoRA’s sparsity) outweighs their benefits. LoRA’s simplicity — train B and A, nothing else — is robust precisely because it has no overhead.

The Task 2 result reinforces the optimizer-theoretic distinction between proximal methods and subgradient methods. This is not merely an engineering detail but a fundamental algorithmic property: exact sparsity requires solving an analytic subproblem (proximal) rather than taking a gradient step (SGD). This has implications beyond SoRA — any sparse learning system that needs true zeros (model compression, pruning, feature selection) should use proximal operators or equivalent analytic solvers.

9 Conclusion

We compared three PEFT methods — LoRA, AdaLoRA, and SoRA — on the CoLA grammatical acceptability task using DeBERTa-v3-base. Our main findings are:

1. **SoRA achieved the best MCC (0.2294)**, followed by LoRA (0.1557) and AdaLoRA (0.0901). Without meaningful sparsity induction, SoRA benefits from additional expressiveness (gate vectors) and extended training.
2. **AdaLoRA underperforms on small datasets.** Its adaptive rank pruning introduces training instability during the pruning window (steps 200–1000), causing loss to increase for the first four epochs. The dataset is too small for the model to amortize this overhead.
3. **SoRA’s proximal sparsity mechanism does not trigger under strong task gradients.** Inducing true sparsity requires λ large enough to overcome the task loss gradient defending gate values — revealing a fundamental performance–sparsity tradeoff.
4. **Proximal gradient produces exact zeros; SGD subgradient cannot.** Our toy experiment confirms this empirically: proximal recovered 5–6 of 6 true zeros exactly, while SGD left residuals of 10^{-3} – 10^{-4} on all components. Both methods converge to nearly identical objective values, confirming the distinction is about solution structure, not optimality.
5. **LoRA’s effective rank analysis reveals heterogeneous adaptation needs.** Layers vary from erank 4.4 to 7.8, motivating adaptive approaches — but in the small-data regime, the instability cost exceeds the efficiency benefit.

References

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
- [2] Q. Zhang, M. Chen, A. Bukharin, N. Karampatziakis, P. He, Y. Cheng, W. Chen, T. Zhao. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. *ICLR*, 2023.
- [3] N. Ding, X. Qin, G. Yang, F. Wei, Z. Yang, Y. Su, S. Hu, Y. Chen, C.-M. Chan, W. Chen, J. Yi, W. Zhao, X. Wang, Z. Liu, H. Zheng, J. Chen, Y. Liu, J. Tang, M. Sun. Parameter-Efficient Fine-Tuning of Large-Scale Pre-Trained Language Models. *Nature Machine Intelligence*, 2023. (SoRA reference)
- [4] P. He, J. Gao, W. Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. *arXiv:2111.09543*, 2021.
- [5] A. Warstadt, A. Singh, S. R. Bowman. Neural Network Acceptability Judgments. *Transactions of the Association for Computational Linguistics*, 7:625–641, 2019.
- [6] K. Clark, M.-T. Luong, Q. V. Le, C. D. Manning. ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. *ICLR*, 2020.
- [7] A. Gu, T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv:2312.00752*, 2023.
- [8] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, S. Hochreiter. xLSTM: Extended Long Short-Term Memory. *arXiv:2405.04517*, 2024.